

SPECHUNTER: A Fail-Closed Agent Loop for Security Discovery and Repair on BOOM RTL

Author names withheld in draft
<https://github.com/Ryclic/SpeechHunter>

Abstract—We present SPECHUNTER, a CHIA loop in which LLM agents propose attack programs, repairs, and follow-up attacks against the BOOM out-of-order RISC-V core, while a deterministic validator decides every verdict from cycle-accurate simulation. The design premise is that an agent’s claims are cheap and its mistakes are plausible, so the loop’s value lies in what it refuses to accept. In a live run, Gemini 2.5 Flash-Lite drove the loop against a Verilator build of SmallBoomV3: it found an intentionally seeded cache leak, minimized it to a three-operation witness, selected a repair, and the validator required a clean retest and attacker exhaustion before accepting it (28 BOOM executions, 4 model calls, \$0.0006). An eight-program held-out corpus then separated the vulnerable and repaired variants in all 64 executions. We also applied the loop to upstream BOOM issue #715 on the exact reported revision. The waveform trace shows that the address first taken as evidence of leakage comes from an unrelated load, while the dependent load is stopped by an existing register-read gate. As a result, the issue did not reproduce, and none of four candidate RTL repairs can be credited. We report these negative results and eleven verifier defects exposed by live runs and their review, because they are the part of this work most likely to be useful to others building agentic hardware-security loops.

I. INTRODUCTION

Transient-execution attacks [1], [2] arise from interactions between speculation, privilege checks, and cache state that are hard to enumerate by hand. LLM agents can write plausible attack programs and RTL patches quickly, which makes them attractive for security red-teaming of open cores. They are also very good at producing *plausible but wrong* conclusions: a repair that “fixes” a leak that never occurred, a minimized witness that no longer exercises the threat, or a clean retest that is clean because the attack became irrelevant.

SPECHUNTER is a CHIA [3] loop for the BOOM core [4] in Chipyard [5], built around one rule: *agents propose, the validator disposes*. Agents never emit executable patches or observations; they select from closed, reviewed program operations and repair identifiers, and every verdict is recomputed from raw simulator output bound by SHA-256 to the exact source, binary, and runner that produced it. Our contributions are:

- A four-agent CHIA loop (recon, attacker, validator, repair) with mandatory witness retest and a strict attacker-exhaustion criterion, running against cycle-accurate BOOM with Spike [6] as the architectural reference and an atomic cost ledger (§II).
- End-to-end evidence on real BOOM RTL, clearly separated from fixture and model-only evidence (§III).

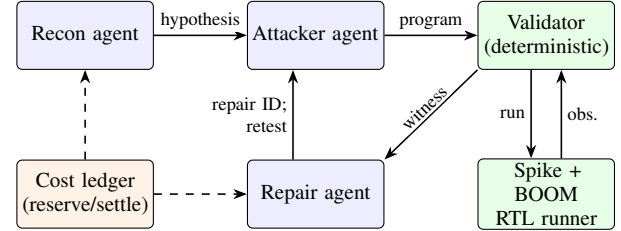


Fig. 1. The SPECHUNTER loop. Blue: LLM agents (Gemini 2.5 Flash-Lite via Vertex AI). Green: trusted, deterministic components that decide every verdict.

- A trace-level investigation of BOOM issue #715 [7] on its historical revision that did *not* reproduce the reported mechanism and identified why (§IV).
- A catalogue of verifier defects found during live runs, each now covered by a regression test (§V).

II. LOOP DESIGN

Agents. Figure 1 shows the loop. The recon agent states a hypothesis against a security invariant (user code must not observe a PMP-protected secret, architecturally or through cache timing). The attacker turns it into a program over a whitelist of operations (e.g., `enter_user`, `train_branch`, `load_secret`, `encode`, `probe`); the trusted runner compiles these to fixed, reviewed RV64 instruction templates. The repair agent may only choose a closed repair ID; it cannot emit patch text.

Validator. Each candidate runs in four executions: two secret values, each repeated twice. A violation requires a deterministic, secret-dependent observation with the expected load-access fault and no architectural secret value; anything else is *inconclusive*, never clean. Findings are minimized by delta debugging, and the minimized program must itself reproduce before it is recorded.

Repair acceptance. After a repair, the orchestrator (not the model) replays the exact minimized witness, which must be clean. Control then returns to the attacker. A repair is verified only on attacker exhaustion, and in the current schema (v5) exhaustion additionally requires a distinct, threat-relevant challenge that violates on the *unrepaired* variant and is clean on the repaired one. Any later violation revokes verification.

Provenance and cost. Chipyard 1.14, BOOM, and toolchains are pinned by commit and hash; the runner refuses dirty trees or mismatched binaries. Every evidence file is bound into a SHA-256 seal that the test suite recomputes. A

TABLE I

SEALED EVIDENCE BY TARGET TYPE. “MUTATION” IS AN INTENTIONALLY SEEDED HARNESS LEAK, NOT AN UPSTREAM BOOM BUG.

Experiment	Target	Result
PMP privilege gate (Spike + BOOM)	BOOM RTL	pass, trap 5, no leak
Pristine baseline matrix	BOOM RTL	8/8 clean
Seeded positive control	BOOM+mutation	8/8 violating; 8/8 clean after repair
Live Vertex loop	BOOM+mutation	found, repaired, exhausted; \$0.0006
Held-out corpus (8 programs)	BOOM+mutation	64/64 correct builds; 8/8 clean; <i>no security claim</i>
LSU fault-gate RTL patch	patched BOOM	not reproduced
Issue #715, current + historical	BOOM RTL	see Table II
Guided vs. random search	Fixture	10/10; \$0.0054 total
10× loop repeatability	Model fixture	complete; \$0.0005
Loop through CHIA node	Model fixture	

locked JSON ledger reserves a conservative cost before each model call and settles it from token metadata, halting at a hard cap. SPECHUNTER runs as a decorated CHIA node on a local Ray runtime.

III. EVALUATION

All BOOM runs used GCP `e2-standard-8` workers with no service account, a six-hour deletion cap, and explicit deletion after evidence recovery. Table I lists every result together with the kind of target it was measured on; we do not combine results across kinds.

Real BOOM baseline. A reviewed payload configures PMP to deny a 4 KiB page, drops to user mode via `mret`, and requires a load-access fault (cause 5). The same ELF passed on Spike and on SmallBoomV3. Two programs (architectural denial and a transient window) were clean and deterministic across secret worlds on the pristine core. Because pristine BOOM exhibited no violation, we added a clearly labelled positive control: a harness variant that touches a secret-selected cache line before entering user mode. Its repair, `remove-seeded-cache-leak`, removes that access. The control produced probe sequences `[1]`, `[0]`, `[1]`, `[0]` before repair and all zeros after, over 16 executions on the *same* simulator binary.

Live agent run. On the positive control, the model proposed a six-operation program; BOOM confirmed the leak; minimization kept `enter_user`, `load_secret`, `probe`. The repair agent chose the closed repair, the witness retested clean, and the attacker then reached exhaustion: 28 BOOM executions, 4 Vertex calls, zero inconclusive cases, and \$0.0006038 of accounted cost. This transcript predates the schema-v5 exhaustion rule and exhausted after only the mandatory replay; under the current rule it would require one further discriminating challenge.

Held-out corpus. Eight fixed programs vary training, encoding, squash and fence placement, and noise, all retaining the protected load. With two repetitions in both secret worlds, all 32 mutated executions were deterministic violations and all 32 repaired executions were clean. This tests that the *validator*

TABLE II

SEARCH ON DETERMINISTIC FIXTURES (PRIVILEGE, TRANSIENT-CACHE, AND SECURE-CONTROL BENCHMARKS; 16-ITERATION BUDGET).

	Guided	Random (1000 seeds)
Positive cases found	2/2	1145/2000 (57.3%)
95% Wilson interval	–	55.1–59.4%
Mean attempts to find	1.5	8.48
False positives (secure control)	0	0/1000

and repair gate generalize across program shape; it does not test an RTL fix.

Candidate RTL patch. Source review of BOOM’s `lsu.scala` found that incoming and retried D-cache requests are not gated by same-cycle `ae_ld/pf_ld/ma_ld`. A two-path gate patch built into a distinct simulator and ran the secure matrix cleanly (8/8). Since the baseline was already clean, this shows only that the patch builds and does not break the target tests. The seal records `security_fix_validated: false`.

Search and repeatability. Table II compares the deterministic guided policy with random programs over three small fixtures. The guided arm is a single deterministic run, so this is a sanity check on search structure, not a statistical claim about LLM agents. Separately, ten independent Gemini-driven trials on a fast model of the positive control all completed discovery, repair, retest, and exhaustion (40 calls, 264 fixture executions, \$0.0053752). A further run through the CHIA `run_agent_experiment` node (CHIA 1.0.1, Ray 2.54.0) completed the same lifecycle for \$0.0005130.

IV. CASE STUDY: BOOM ISSUE #715

Issue #715 [7] reports that on a historical BOOM a supervisor-page load that faults can still forward data to a dependent load during speculation, leaking it through the TLB. We tested it three ways.

Adaptation. A reviewed adaptation of the mechanism was deterministic and clean on the current pin (4 executions) and on the exact reported Chipyard `004297b6/BOOM fac2c370` with historical `SmallBoomConfig` (built in 685 s).

Original attachment. We fetched the reporter’s ELF by pinned URL and digest, converted its entry-zero image to a fixed load image, and captured a VCD waveform at a fixed seed. Table III summarizes the gadget. A scanner joins events by ROB index, physical register, load-queue slot, and branch mask, and requires one ordered chain within the first speculation window. The trace shows the trained branch (cycle 3769), a branch-masked protected-page TLB request (3807), requests to `0x59f` (from 3809), a page fault, and misprediction resolution (3910). At first sight this looks like the reported leak. However, the disassembly shows that `0x59f` is the address of an independent third load (`lb s1, 1439(a0)` with a zero base; `1439 = 0x59f`), not a secret-derived address. The dependent load at gadget offset +4 issues with its source operand poisoned while the LSU reports a load miss, and

TABLE III
ISSUE #715 ATTACHMENT GADGET (WRONG PATH) AND WHAT THE HISTORICAL-RTL WAVEFORM SHOWS FOR EACH LOAD.

Offset	Instruction	Observed in waveform
+0	lb sp, -2048(t1)	protected-page TLB req. (3807); fault
+4	ld s1, 0(sp)	issues poisoned; register read gated
+8	lb s1, 1439(a0)	requests 0x59f (from 3809)

TABLE IV
VERIFIER DEFECTS FOUND AND CLOSED. FP = WOULD HAVE ENABLED A FALSE POSITIVE (A FALSE FINDING OR FALSE “FIXED”).

Defect	Effect
Minimizer could drop the protected load from a witness	FP finding
Retest of the same program counted toward exhaustion	FP fix
Irrelevant (e.g. probe-only) challenge counted as clean	FP fix
Challenge clean on <i>both</i> variants counted as clean	FP fix
Verified flag survived a later-cycle violation	FP fix
Stale clean-replay flag carried across recon cycles	FP fix
Short observation batch skipped repeatability check	FP either
VCD scanner read intermediate combinational values	FP finding
Unrelated trace events could form a “mechanism chain”	FP finding
Seed equality inferred from matching timing	provenance
Git porcelain <code>.strip()</code> rejected valid repair tree	false reject

BOOM’s register-read gate (`exu/core.scala`, lines 973–978) drops its issue-valid, so it never presents a valid LSU request.

Repairs. Before this attribution was clear, we iterated on four hash-pinned candidate LSU patches outside the closed repair list, each driven by the previous matched-trace comparison: v1 same-cycle fault gates; v2 blocking loads under unresolved branches; v3 suppressing fast wakeup and excepting-entry writeback; v4 requiring a registered D-cache fire before speculative wakeup. V1–v3 left the 0x59f chain cycle-identical to baseline, and v4’s waveform was not recovered. Because the baseline never demonstrated the dependent-load mechanism, *no* repair can be credited, and every comparison seal records `security_fix_validated: false`. A loop that accepted “the patch built and nothing leaked” would have declared success four times here.

V. WHAT THE LIVE RUNS TAUGHT US

Table IV lists verifier defects exposed by live runs or review of their transcripts. Each would have produced an unsupported claim. Each is now covered by a regression test.

Three lessons generalize. First, *a clean result is only evidence if the same test fails on the vulnerable variant*; four of the defects are versions of this. Second, minimizers and scanners are adversarial surfaces: they optimize for the property check, not the threat model, and must re-encode the threat model as constraints. Third, a negative result on real RTL (issue #715) required more infrastructure than the positive control, and it is also the more informative of the two.

VI. LIMITATIONS AND RELATED WORK

We found no new BOOM vulnerability; every successful discovery and repair is on an intentional harness mutation.

The live BOOM loop ran once; repeatability is shown on a model fixture only. The search comparison uses three toy fixtures. Repairs are chosen from a closed list, so the agent diagnoses rather than synthesizes RTL, a deliberate trade of generality for soundness. The cache observer covers two lines in a fixed order. Coverage-directed RTL fuzzing [8] and speculation contracts [9] offer complementary search and specification methods that could replace our fixed operation set and invariant.

VII. CONCLUSION

SPECHUNTER shows that an LLM-driven security loop on real out-of-order RTL is cheap to run (under a tenth of a cent per live cycle) and that its credibility comes from a validator that fails closed. The loop, pinned BOOM runners, and every sealed artifact are open source at <https://github.com/Ryclic/Sp ecHunter>.

AI ASSISTANCE DISCLOSURE

Gemini 2.5 Flash-Lite is a component of the evaluated loop. AI coding assistants (OpenAI Codex, Google Gemini/Antigravity, and Anthropic Claude Code) were used to write and review code and tests, analyze evidence, and draft and edit this paper. The human authors reviewed all claims against the sealed evidence and are responsible for the paper’s content.

REFERENCES

- [1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, “Spectre attacks: Exploiting speculative execution,” in *IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, “Meltdown: Reading kernel memory from user space,” in *USENIX Security Symposium*, 2018.
- [3] UC Berkeley Architecture Research, “CHIA agentic co-design framework,” <https://github.com/ucb-bar/chia>, commit 16c35e92, 2026.
- [4] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, and K. Asanović, “BOOM v2: An open-source out-of-order RISC-V core,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2017-157, 2017.
- [5] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, “Chipyard: Integrated design, simulation, and implementation framework for custom SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [6] RISC-V International, “Spike RISC-V ISA simulator,” <https://github.com/riscv-software-src/riscv-isa-sim>, 2024.
- [7] T. Kovats, “[BUG REPORT] speculative meltdown-type leakage through the TLB, riscv-boom issue #715,” <https://github.com/riscv-boom/riscv-boom/issues/715>, 2025.
- [8] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, “RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.
- [9] M. Guarnieri, B. Köpf, J. Reineke, and P. Vila, “Hardware-software contracts for secure speculation,” in *IEEE Symposium on Security and Privacy (S&P)*, 2021.